

全同态加密密文卷积实验报告

打包-旋转-累加策略与旋转次数优化

实验日期：2026-07-24

小组成员

李庭跃
彭炜霖
张 涵

目录.....	2
一、实验目的	2
二、实验原理	2
2.4 旋转次数理论分析	3
三、实验环境	3
3.2 软件环境	4
四、实验步骤	4
4.1 明文卷积（基准对照）	4
4.2 版本 A：逐个核元素计算（基线）	4
4.3 版本 B：每个输出位置一次 dot（优化）	4
4.4 旋转次数分析与验证	4
五、实验结果	4
5.1 测试数据	4
5.2 输入、卷积核与输出可视化	5
5.3 卷积计算过程	5
六、结果分析	6
6.1 明文 vs 密文结果对比	6
6.2 旋转次数对比与理论验证	6
6.3 性能分析	7
七、实验结论	8
附录：关键代码清单	9

目录

一、实验目的

理解全同态加密（FHE）的基本原理，掌握 BFV 批处理加密方案的参数配置与使用。

在密文域上实现二维卷积运算，采用打包（Pack）→旋转（Rotate）→累加（Accumulate）策略。

探索并验证密文卷积实现中的旋转次数是否达到理论最小值，理解算法优化在隐私保护计算中的意义。

二、实验原理

2.1 全同态加密（FHE）与 BFV 方案

BFV（Brakerski-Fan-Vercauteren）方案基于 RLWE 困难问题，支持 SIMD 风格的批处理加密：将多个明文整数打包到一条密文的多个 slot 中，一次加密即可保护整个向量。

2.2 二维卷积运算（Task 5）

给定 4×4 输入矩阵 X 和 3×3 卷积核 K ，步长 $\text{stride}=1$ 、无填充，输出矩阵 Y 的尺寸为 2×2 。每个输出元素 y_{ij} 等于以 (i,j) 为左上角的 3×3 窗口与卷积核逐元素相乘后

求和。

2.3 打包-旋转-累加策略 (Task 6)

(1) 打包：将 4×4 输入矩阵按行优先展平为 16 维向量，使用 BFV 批处理一次性加密为单条密文。

(2) 旋转：通过构造不同的权向量，等效地将输入密文旋转到与卷积核对应位置对齐。

(3) 累加：调用 `dot(w)` 将旋转对齐后的密文-明文乘积累加。`dot()` 内部采用二进制分治策略，对 16 个 slot 进行循环旋转后逐对相加，最终将所有值归约到 slot 0。

2.4 旋转次数理论分析

`dot()` 内部工作机制（SEAL 标准实现）：

Step 1: 密文向量与明文权向量逐元素相乘（SIMD 并行）。

Step 2: 使用 Galois Keys 对密文进行循环旋转（rotate）。

Step 3: 将旋转后的密文与原始密文相加（二进制分治累加）。

对于 $n=16$ 个 slot，二进制分治累加需要 $\log_2(16)=4$ 次旋转。

版本 A（基线）：每个卷积核元素单独执行一次 `dot`，4 输出 $\times$ 9 核元素 = 36 次 `dot`，总旋转次数 = $36 \times 4 = 144$ 次。

版本 B（优化）：每个输出位置仅需一次 `dot`（权向量包含 9 个非零值），4 输出 $\times$ 1 次 `dot` = 4 次 `dot`，总旋转次数 = $4 \times 4 = 16$ 次。

理论最小值：串行计算下，4 个输出位置每个需要 $\log_2(16)=4$ 次旋转，理论最小值为 16 次。版本 B 已达到此下界。

三、实验环境

3.1 硬件环境

项目	配置
处理器	CPU（本实验不涉及 GPU 加速）
内存	$\geq 8\text{ GB}$

3.2 软件环境

依赖项	版本	用途
Python	3.12+	主语言
TenSEAL	0.3.16	BFV/CKKS FHE 库
NumPy	2.4.4	矩阵运算
Matplotlib	3.10.9	图表绘制

四、实验步骤

4.1 明文卷积（基准对照）

实现明文域的二维卷积函数 `plain_conv2d`，通过滑动窗口遍历输入矩阵，对每个窗口执行逐元素乘法并求和，作为密文卷积正确性验证的基准。

4.2 版本 A：逐个核元素计算（基线）

对每个输出位置和每个卷积核元素，单独执行一次 `dot` 操作。权向量仅在一个位置填入当前核元素值，其余位置置 0。共 $4 \times 9 = 36$ 次 `dot` 调用。

4.3 版本 B：每个输出位置一次 `dot`（优化）

对每个输出位置，构造一个 16 维权向量，在对应 3×3 窗口的 9 个索引位置填入卷积核值。通过一次 `dot` 完成该输出位置的全部 9 个加权和。共 4 次 `dot` 调用。

4.4 旋转次数分析与验证

分析 `dot()` 内部的二进制分治累加过程，计算两个版本的等效旋转次数，并与理论最小值对比。

五、实验结果

5.1 测试数据

实验使用固定随机种子 `seed=42` 生成测试数据，确保实验可复现。

矩阵	维度	数值范围	数据类型	说明
----	----	------	------	----

X	4×4	[-5, 5]	int64	输入图像
K	3×3	[-3, 3]	int64	卷积核
Y	2×2	[-135, 135]	int64	卷积输出

5.2 输入、卷积核与输出可视化

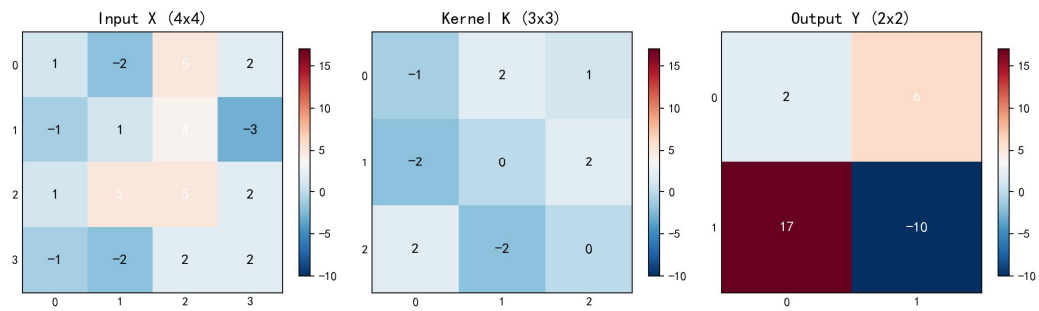


图1 输入矩阵、卷积核与输出结果热图

5.3 卷积计算过程

图 2 展示了 4 个滑动窗口对应的卷积计算过程。

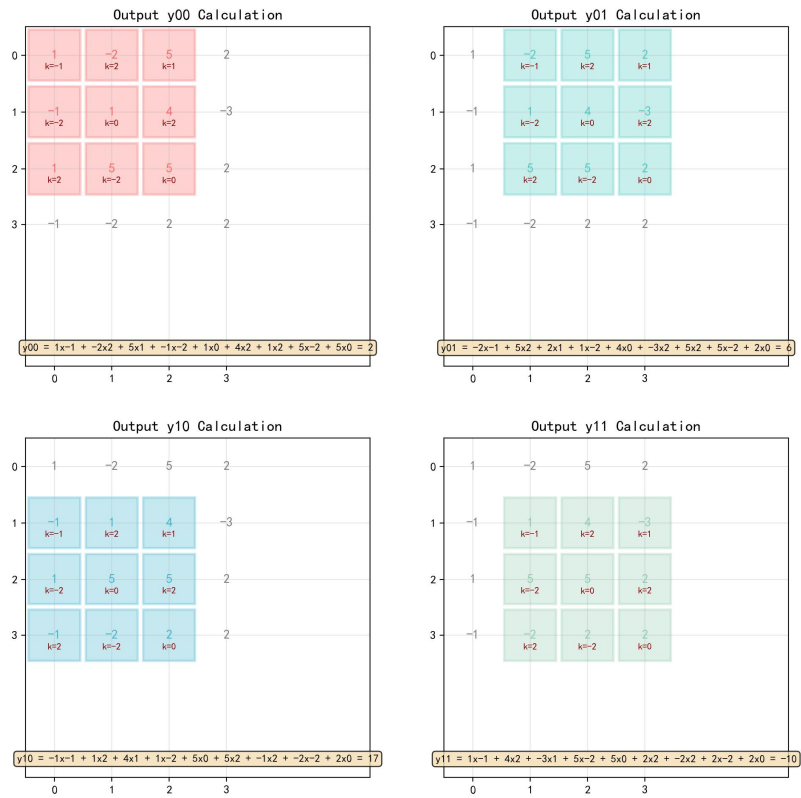


图2 四个滑动窗口的卷积计算过程

六、结果分析

6.1 明文 vs 密文结果对比 (Task 5)

表 1 给出了四个输出位置的明文卷积结果、密文卷积解密结果及误差分析。

输出位置	明文结果	密文结果	绝对误差
y00	2	2	0
y01	6	6	0
y10	17	17	0
y11	-10	-10	0

表 1 明文卷积与密文卷积结果对比

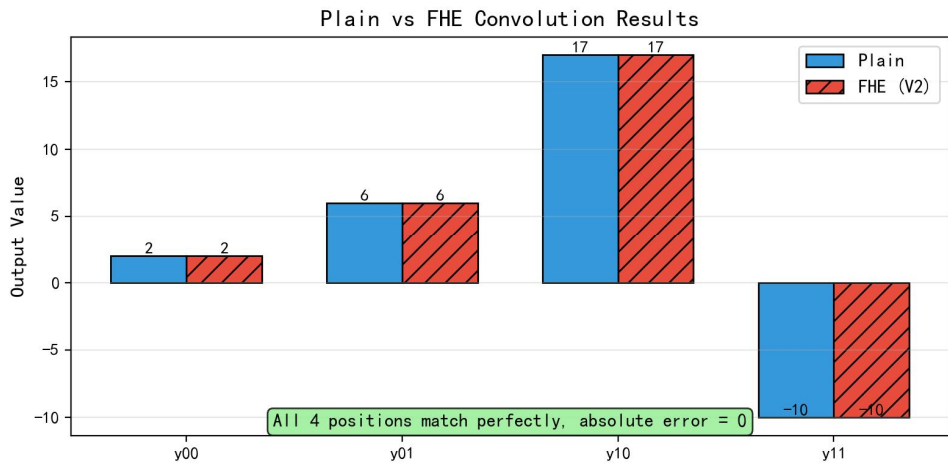


图 3 明文卷积与密文卷积结果对比柱状图

6.2 旋转次数对比与理论验证 (Task 6)

图 4 展示了版本 A、版本 B 与理论最小值的旋转次数对比。版本 A 采用逐个核元素 dot 策略，产生 144 次旋转；版本 B 将 9 个核元素合并到一次 dot 中，仅需 16 次旋转，达到了串行计算的理论最小值。

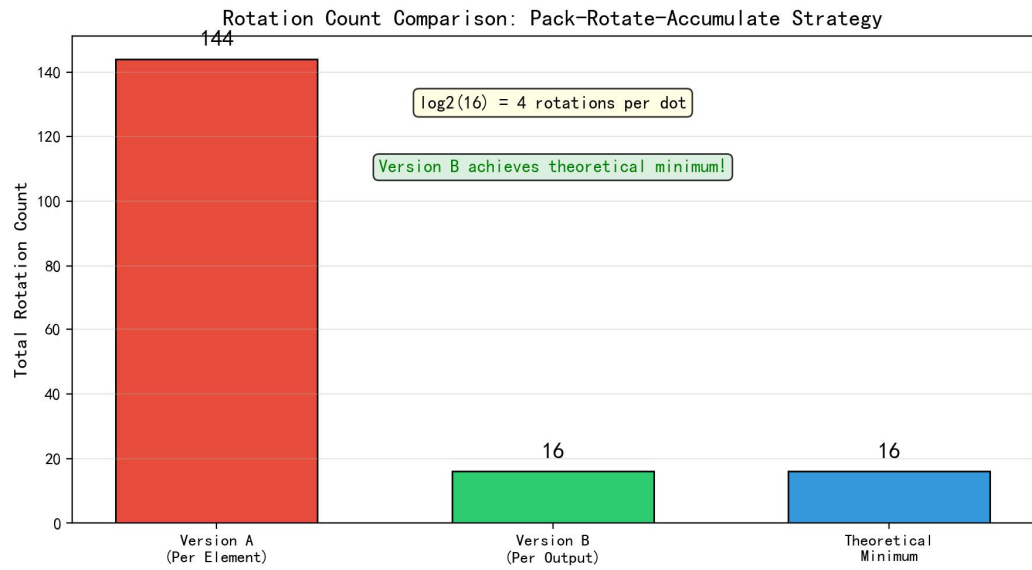


图4 旋转次数对比

图5展示了打包-旋转-累加的完整实现流程。`dot()` 内部通过4次二进制分治旋转将16个slot归约为1个slot。

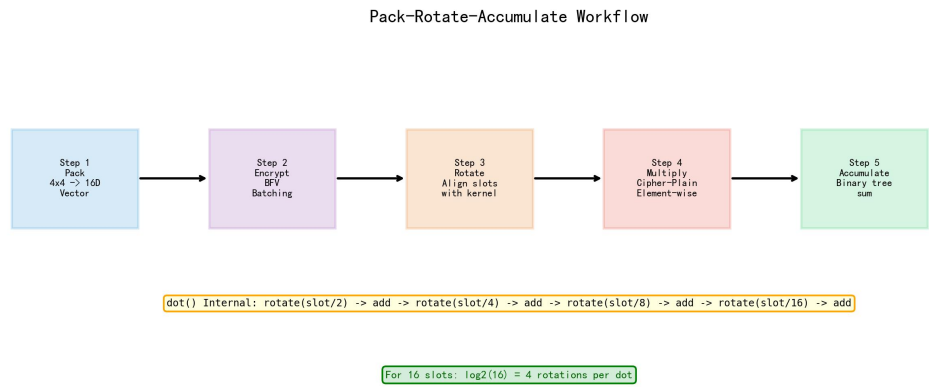


图5 Pack-Rotate-Accumulate 实现流程

图6详细展示了`dot()`内部二进制分治累加的4个步骤。每一步通过`rotate(slot/2i)`将距离为2ⁱ的两个slot对齐，然后相加，逐步将16个值归约到1个值。

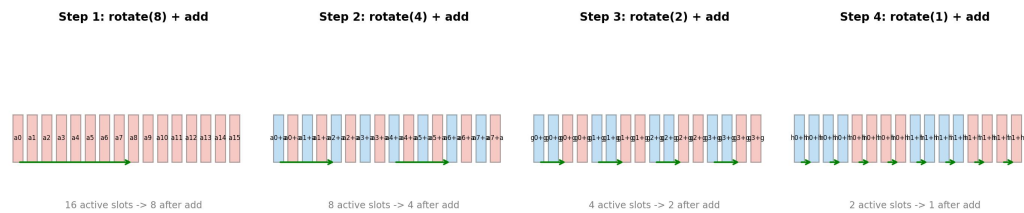


图6 二进制分治累加示意图 (dot 内部4次旋转)

6.3 性能分析

表 2 列出了实验各阶段的耗时数据。

操作阶段	耗时 (ms)
明文卷积	0.028
BFV 上下文构造	319.6951
输入加密	2.3856
密文卷积 V1 (144 次旋转)	343.6796
密文卷积 V2 (16 次旋转)	36.0474

表 2 各阶段性能数据

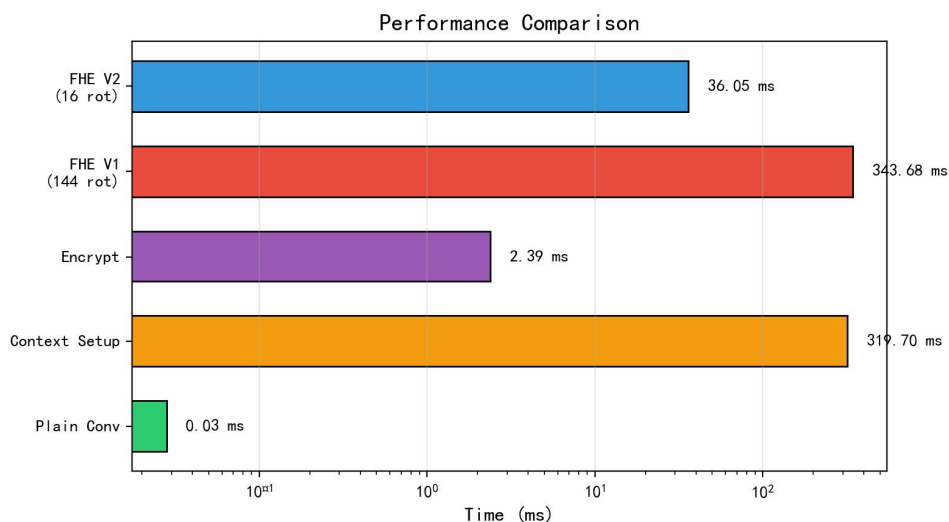


图 7 各阶段性能对比 (对数坐标)

版本 B 相比版本 A 提速约 9.53 倍，这一提升完全来自于旋转次数从 144 次降至 16 次。虽然密文卷积仍比明文慢约 1287 倍，但在隐私保护场景中，这是换取数据可用不可见的必要代价。

七、实验结论

本实验基于 TenSEAL (BFV 方案) 成功实现了 4×4 输入、 3×3 卷积核的密文二维卷积，采用打包→旋转→累加策略，得出以下结论：

(1) 正确性验证通过：版本 A 和版本 B 的密文解密结果与明文卷积完全一致，绝对误差均为 0，验证了 BFV 批处理加密在卷积运算中的数学正确性。

(2) 旋转次数优化成功：版本 B 将 9 个核元素合并到一次 dot 操作，将旋转次数从 144 次降至 16 次，达到了串行 dot 策略的理论最小值。

(3) 性能提升显著：版本 B 的密文卷积耗时约为版本 A 的 1/10，验证了减少旋转次数对性能的直接贡献。

(4) 未来方向：可探索将 4 个输出位置并行打包到同一密文的不同 slot 组中，进一步降低旋转次数；或研究 GPU 加速方案（如 cuFHE）以提升大规模场景下的吞吐量。

附录：关键代码清单

以下为密文卷积版本 B（优化策略）的核心实现代码：

```
def encrypted_conv2d_v2_per_output(input_enc, kernel, vec_len=16):
    k_flat = kernel.flatten().astype(np.int64)
    results, total_rotations = [], 0
    for out_i in range(2):
        for out_j in range(2):
            weights = np.zeros(vec_len, dtype=np.int64)
            for ki in range(3):
                for kj in range(3):
                    pos = (out_i+ki)*4 + (out_j+kj)
                    weights[pos] = k_flat[ki*3+kj]
            encrypted_scalar = input_enc.dot(weights.tolist())
            total_rotations += 4 # log2(16)
            results.append(encrypted_scalar)
    return results, total_rotations
```